arXiv:2306.03509v1 [eess.AS] 6 Jun 2023

Mega-TTS: Zero-Shot Text-to-Speech at Scale with Intrinsic Inductive Bias

Ziyue Jiang [*†♠♡] **Yi Ren** [*♡] **Zhenhui Ye** [*†♠♡] **Jinglin Liu** [♡] **Chen Zhang** [†♡♠]
Qian Yang [♠] **Shengpeng Ji** [♠] **Rongjie Huang** [♠] **Chunfeng Wang** [♡]
Xiang Yin [♡] **Zejun Ma** [♡] **Zhou Zhao** [‡♠]
[♠]Zhejiang University & [♡]ByteDance
ziyuejiang@zju.edu.cn, ren.yi@bytedance.com, zhaozhou@zju.edu.cn

Abstract

Scaling text-to-speech to a large and wild dataset has been proven to be highly effective in achieving timbre and speech style generalization, particularly in zero-shot TTS. However, previous works usually encode speech into latent using audio codec and use autoregressive language models or diffusion models to generate it, which ignores the intrinsic nature of speech and may lead to inferior or uncontrollable results. We argue that speech can be decomposed into several attributes (e.g., content, timbre, prosody, and phase) and each of them should be modeled using a module with appropriate inductive biases. From this perspective, we carefully design a novel and large zero-shot TTS system called Mega-TTS, which is trained with large-scale wild data and models different attributes in different ways: 1) Instead of using latent encoded by audio codec as the intermediate feature, we still choose spectrogram as it separates the phase and other attributes very well. Phase can be appropriately constructed by the GAN-based vocoder and does not need to be modeled by the language model. 2) We model the timbre using global vectors since timbre is a global attribute that changes slowly over time. 3) We further use a VQGAN-based acoustic model to generate the spectrogram and a latent code language model to fit the distribution of prosody, since prosody changes quickly over time in a sentence, and language models can capture both local and long-range dependencies. We scale Mega-TTS to multi-domain datasets with 20K hours of speech and evaluate its performance on unseen speakers. Experimental results demonstrate that Mega-TTS surpasses state-of-the-art TTS systems on zero-shot TTS, speech editing, and cross-lingual TTS tasks, with superior naturalness, robustness, and speaker similarity due to the proper inductive bias of each module. Audio samples are available at `https://mega-tts.github.io/demo-page`.

1 Introduction

Text-to-speech (TTS) synthesis [53, 2, 49, 35, 48, 45, 29, 66, 43, 28] aims to generate human-like speech from text and has gained significant attention in the field of machine learning. Traditional TTS systems [13, 11, 60, 8, 21] are usually trained on limited datasets, which impairs their models' ability to produce diverse and generalizable results. In contrast, large-scale TTS systems [58, 67, 27] are trained on tens of thousands of hours of speech data, which significantly improves their zero-shot capability [58, 67]. Current large-scale TTS systems typically encode the speech waveform into latent

[*]Equal contribution.
[†]Interns at ByteDance.
[‡]Corresponding author.

Preprint. Under review.

Table 1: The illustration of intrinsic properties for different components in human speech.

Modality	Components	Intrinsic Properties	Suitable for LM
Human speech	Phase	Highly dynamic, irrelevant to semantics	✗
	Timbre	Global and stable	✗
	Prosody	Long-term dependencies Rapid changes Weak relation with text	✓
	Content	Monotonic alignment	✗

with neural codec models [14] as the intermediate representation and model it with autoregressive language models (LM) [58] or diffusion models [50].

As presented in Table 1, human speech can be decoupled into several attributes: content, timbre, prosody, phase, etc. However, current large-scale TTS systems directly use neural audio codec models to encode the entire speech into latent and ignore the following intrinsic nature of speech: 1) phase is highly dynamic and irrelevant to semantics, which means people are much less sensitive to perceive phase than to prosody and timbre, especially for monaural audio. Therefore, only one reasonable phase is needed for waveform reconstruction, and it is not necessary to model all possible phases. Modeling phase with LM or diffusion model can waste a lot of model parameters since they model the full distribution of phase[4]. 2) Timbre should remain stable within the sentence as a global vector. Modeling timbre with time-varying latent is costly[5]. 3) Prosody typically has both local and long-term dependencies and changes rapidly over time with a weak correlation to text, which makes conditional phoneme-level LLMs inherently ideal for generating prosody sequences. 4) Content has monotonic alignment with speech while the autoregressive language model cannot guarantee that, which can lead to repeating or missing word issues [59, 58, 67].

To make use of the large and wild training datasets while matching the inductive bias of the model and the intrinsic nature of speech, we propose a zero-shot text-to-speech model called Mega-TTS. Specifically, 1) considering the limitations of neural audio codec models, we select mel-spectrogram as the intermediate representation to separate the phase and other attributes. We adopt a GAN-based vocoder to reconstruct the phase information to improve our model's efficiency. 2) To model timbre information, we employ global vectors since timbre is a global attribute that changes slowly over time. We extract the global information from a different speech of the same speaker with the global speaker encoder to decompose the timbre and content information. 3) To capture prosody information in a sentence, we adopt a VQGAN-based acoustic model to generate the mel-spectrogram and a latent code language model called P-LLM to fit the distribution of prosody. The P-LLM is capable of capturing both local and long-range dependencies for prosody modeling.

To evaluate the zero-shot performance of Mega-TTS, we perform experiments on VCTK [57], AISHELL-3 [51] and LibriSpeech test-clean [42] datasets. All of the test speakers are unseen in the training corpus. Our Mega-TTS surpasses the state-of-the-art zero-shot TTS systems [8, 58] in terms of speaker similarity, speech naturalness, and generation robustness, which demonstrates the superiority of introducing appropriate inductive biases. Moreover, Mega-TTS outperforms state-of-the-art models on speech editing [52, 3] and cross-lingual TTS [67] tasks. The main contributions of this work are summarized as follows:

- We propose Mega-TTS, a zero-shot text-to-speech system that considers intrinsic inductive biases. Instead of using latent encoded by audio codec as the intermediate representation [64, 14, 58], we decompose mel-spectrogram into content, timbre, prosody, and phase attributes and model each of them according to their intrinsic properties.

[4]That is why GAN-based vocoders [31] are popular.

[5]Our method retains a small portion of time-varying timbre information in the latent code, while the majority is represented as the global vector.

- We train Mega-TTS on a multi-domain and multi-lingual dataset that contains 20k hours of speech data. It is worth noting that existing large-scale TTS systems [58, 50] are typically trained with speech corpora from audiobooks, while our system is trained on multi-domain speech corpora.
- We evaluate Mega-TTS on 3 down-stream speech generation tasks (i.e., zero-shot TTS, speech editing, and cross-lingual TTS), demonstrating that Mega-TTS can be applied to various speech generation tasks. We also propose a novel sampling strategy for speech editing via the discrete prosody tokens extracted by Mega-TTS.

2 Background

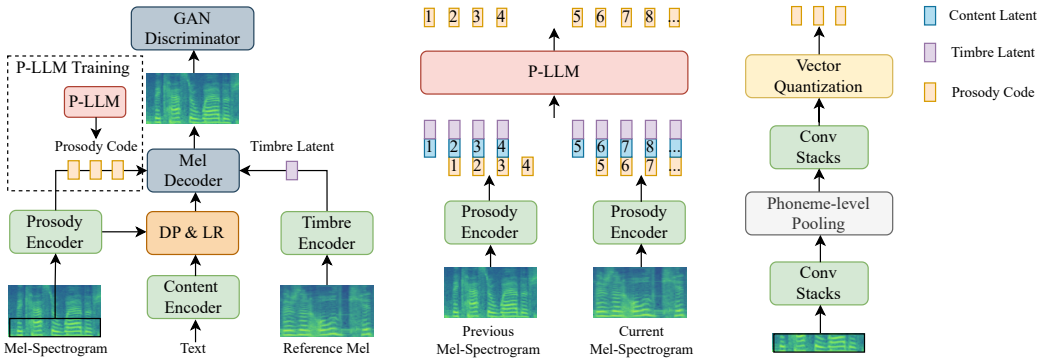
In this section, we briefly overview the background of this work, including zero-shot text-to-speech (TTS) and generative models for speech synthesis.

Zero-shot text-to-speech. Text-to-speech models usually generate mel-spectrogram from text [59, 2, 35, 48, 29, 47, 36, 22] and then synthesize speech waveform from the generated mel-spectrogram using a separately pre-trained vocoder [41, 31, 62, 20], or directly generate waveform from text in an end-to-end manner [45, 15, 30, 37]. For decades, the increasing demand for personalized speech generation in various applications has posed challenges for TTS models [53], especially in zero-shot multi-speaker scenarios regarding domain shifts. Previous approaches can be categorized into speaker adaptation [13, 11, 60, 23] and speaker encoding [25, 1, 26, 61] methods. Traditional works are typically trained on small datasets [11, 23, 21, 8], while some recent works [4, 58, 27, 67] are trained on large-scale datasets and demonstrate the effectiveness in zero-shot scenarios. These systems utilize the neural audio codec models [64, 14] to convert audio waveform into latent and consider it as the intermediate representation for speech generation. Among them, SPEAR-TTS [27] splits the TTS task into two sequence-to-sequence tasks, which enables the training using abundant audio-only data. NaturalSpeech 2 [50] uses a text-conditioned diffusion model to generate the latent vectors of the neural audio codec model. VALL-E [58, 67] proposes the first neural codec language model for text-to-speech, exhibiting strong in-context learning abilities to overcome challenges in zero-shot speech generation. However, these methods ignore the intrinsic property of speech and may lead to inferior or uncontrollable results (e.g., word skipping, repeating, and collapse [58, 67]). Considering the nature of different speech attributes, the autoregressive language model is ideally suitable for prosody modeling. ProsoSpeech [46] has proposed to improve the prosody modeling for TTS with latent prosody vectors predicted by a language model. Nevertheless, it lacks the in-context learning capacity, which restricts its application scenarios.

Generative models for speech synthesis. Generative models, like language models [4, 33], VAE [34, 47], GAN [31, 30], Normalizing flow [39, 29], and diffusion model [32, 24, 43, 22], have been applied to speech or audio synthesis for years. Previous works of autoregressive generative model mainly aim at waveform generation [41, 18] and continuous acoustic feature generation [59, 49]. Recently, speech generation systems like AudioLM [4] and VALL-E [58] propose to utilize neural audio codec models [64, 14] to convert audio waveform into discrete codes as the intermediate representation and design LLMs to generate these codes to achieve speech synthesis. Although good reconstruction quality can be achieved by neural audio codec models, they ignore the intrinsic nature of speech [14] and may not be suitable to serve as the generator of intermediate representation for speech generation. The encoded latent contains the phase, content, and timbre attributes and language models are not suitable for predicting these due to the error propagation problem.

3 Method

To introduce proper inductive biases into large-scale TTS systems, we propose Mega-TTS, a zero-shot TTS system for natural and robust speech generation in various scenarios (i.e., zero-shot prompt-based TTS, speech editing, and cross-lingual TTS). As shown in Figure 1, Mega-TTS consists of a VQGAN-based [16] TTS model and a prosody large language model (P-LLM). We carefully model different speech attributes in different ways. First, we choose the mel-spectrogram as the intermediate representation as it separates the phase from other attributes very well. Secondly, we extract the global vector from the random previous sentence of the same speaker with the global timbre encoder to disentangle the timbre and content information. Finally, we further use a VQGAN-based acoustic



(a) Mega-TTS (b) P-LLM Training (c) Prosody Encoder

Figure 1: The overall architecture for Mega-TTS. In subfigure (a), P-LLM denotes the prosody large language model; DP & LR denote the duration predictor and length regulator proposed in FastSpeech [48]. In subfigure (b), P-LLM autoregressively predicts the discrete prosody codes.

model to generate the mel-spectrogram and propose a latent code language model called P-LLM to fit the distribution of prosody, since language models are capable of capturing both local and long-range dependency. During inference, we propose to use the content from the given text sequence, the timbre extracted from the prompt speech, and the prosody predicted by our P-LLM to generate the target speech, which is a novel TTS decoding mechanism called *prosody-oriented speech decoding*. Finally, to demonstrate that our model can be applied to various scenarios, we design inference strategies for downstream tasks. We describe these designs and the training and inference procedures in detail in the following subsections.

3.1 Disentangling speech into different components

To introduce appropriate inductive biases into different speech attributes, we need to separately express these attributes and carefully design different architectures for them. The overall model architecture of Mega-TTS is shown in Figure 1. We use three types of encoders to separately encode content, prosody, and timbre representations. Then we adopt a GAN-based mel-spectrogram decoder to generate mel-spectrograms with these representations. We describe the disentangling strategy and detailed design of the proposed encoders as follows.

Disentangling strategy. We disentangle the mel-spectrogram into content, prosody, and timbre representations with the reconstruction loss of the autoencoder and a carefully designed bottleneck [44]: 1) we feed the mel-spectrogram into the prosody encoder, and we also introduce carefully-tuned dimension reduction and phoneme-level downsampling to the prosody encoder to constrain the information flow; 2) the content encoder encodes the phoneme sequence into the content representation; 3) we feed the reference mel-spectrogram sampled from a different speech of the same speaker to disentangle the timbre and content information and temporally average the output of the timbre encoder to get a one-dimensional global timbre vector. The correctly-designed bottleneck will learn to remove the content information and the global timbre information from the output of the prosody encoder, which ensures the performance of disentanglement. Due to the limited page space, we put more details about the hyperparameter selection for the information bottleneck in Appendix D.

Architecture design of encoders. 1) *The prosody encoder* consists of two convolution stacks, a phoneme-level pooling layer, and a vector quantization (VQ) bottleneck. The first convolution stacks compress mel-spectrograms into phoneme-level hidden states according to the phoneme boundary and the second stacks capture phoneme-level correlations. The vector quantization layer [54] then utilizes these hidden states to obtain phoneme-level prosody codes $\mathbf{u} = \{u_1, u_2, ..., u_T\}$ and hidden states $H_{prosody}$. To ease the difficulty of disentanglement, only the low-frequency band of the mel-spectrogram (the first 20 bins in each mel-spectrogram frame) is used as input, as it contains almost complete prosody and significantly less timbre/content information compared to the full band [46]; 2) *The content encoder* is composed of several feed-forward Transformer layers. To achieve the monotonic alignment between the speech content and generated speech, we adopt

the duration predictor and length regulator following common practice in non-autoregressive TTS systems [48, 50]. Differently, we feed the prosody information extracted by the prosody encoder to the duration predictor in order to ease the one-to-many mapping problem [48, 45]; 3) *The timbre encoder* is designed to extract a global vector H_{timbre} that contains the speaker identity of the given speech. The timbre encoder consists of several stacks of convolution layers. To ensure the stability of timbre information across the time axis, we temporally average the output of the timbre encoder to get a one-dimensional timbre vector H_{timbre}.

To keep good perceptual quality, we introduce a GAN-based mel-spectrogram decoder. We adopt the multi-length discriminator [10, 63] based on random windows of different lengths as the discriminator. Overall, the first-stage training loss $\mathcal{L}$ of Mega-TTS can be formulated as:

$$\mathcal{L}_{\mathrm{VQ}} = \|y_t - \hat{y}_t\|^2 + \|\mathrm{sg}[E(y_t)] - z_{\mathbf{q}}\|_2^2 + \|\mathrm{sg}\left[z_{\mathbf{q}}\right] - E(y_t)\|_2^2 \ , \tag{1}$$

$$\mathcal{L} = \mathbb{E}\left[\mathcal{L}_{\mathrm{VQ}} + \mathcal{L}_{\mathrm{Adv}}\right] \ , \tag{2}$$

where y_t is the target speech and $\hat{y}_t$ is the generated speech. $\mathcal{L}_{\mathrm{rec}} = \|y_t - \hat{y}_t\|^2$ is the reconstruction loss, $\mathrm{sg}[\cdot]$ denotes the stop-gradient operation, and $z_{\mathbf{q}}$ is the temporal collection of codebook entries. $\mathcal{L}_{\mathrm{VQ}}$ is the VQVAE loss function [54, 16] and $\mathcal{L}_{\mathrm{Adv}}$ is the LSGAN-styled adversarial loss [38] whose objective is to minimize the distribution distance between the predicted mel-spectrograms and the ground truth mel-spectrograms.

3.2 P-LLM

The P-LLM is a latent code language model that captures local and long-range dependency for prosody modeling. We describe the prosody-oriented speech decoding mechanism and details of the P-LLM as follows.

Prosody-oriented speech decoding. Denote $(\mathbf{y_p}, \mathbf{x_p})$ and $(\mathbf{y_t}, \mathbf{x_t})$ as the prompt and target speech-transcription pairs. Our goal is to synthesize the high-quality target speech $\mathbf{y_t}$ given an unseen speech prompt $\mathbf{y_p}$. During inference, the timbre of the target speech $\tilde{H}_{timbre}$ is expected to be the same as that of the prompt speech. Therefore, to generate the target speech $\mathbf{y_t}$, we only need the prosody information $\tilde{\mathbf{u}}$ of the target speech. Therefore, the prosody-oriented speech decoding procedure can be formulated as follows:

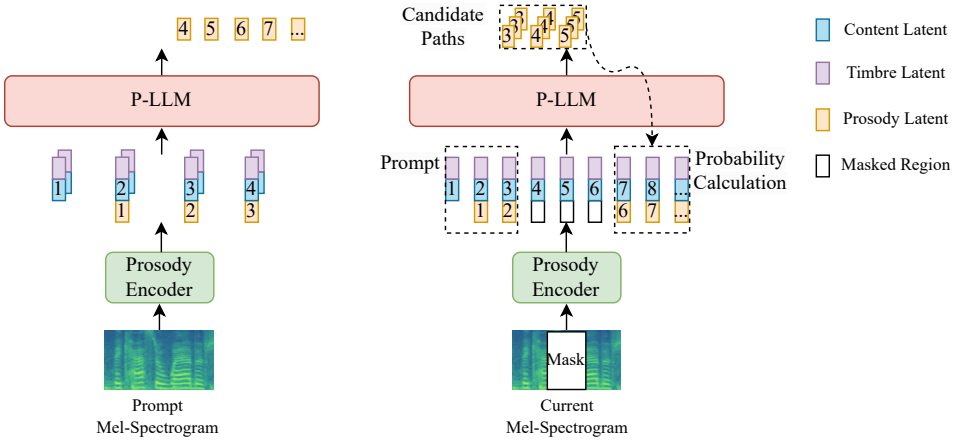
$$\begin{aligned}
&\textbf{Encode}: \mathbf{u} = E_{prosody}(\mathbf{y_p}),\ H_{content} = E_{content}(\mathbf{x_p}),\ \tilde{H}_{timbre} = E_{timbre}(\mathbf{y_p}),\\
&\qquad \tilde{H}_{content} = E_{content}(\mathbf{x_t}) \ ,\\
&\textbf{Prosody prediction}: \tilde{\mathbf{u}} = f(\tilde{\mathbf{u}}|\mathbf{u}, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}; \theta) \ ,\\
&\textbf{Decode}: \hat{y}_t = D(\tilde{\mathbf{u}}, \tilde{H}_{timbre}, \tilde{H}_{content}) \ ,
\end{aligned} \tag{3}$$

where $E_{prosody}$, E_{timbre}, $E_{content}$, and D denote the prosody encoder, timbre encoder, content encoder, and mel decoder. $\mathbf{u}$ is the prosody tokens of the prompt speech, $\tilde{\mathbf{u}}$ is the predicted prosody tokens of the target speech, f is the prosody prediction function, and θ is the parameter of the P-LLM. $\hat{y}_t$ is the generated speech.

Generating prosody codes. The proposed prosody-oriented speech decoding mechanism requires the predicted prosody codes $\tilde{\mathbf{u}}$ of the target speech. Leveraging the powerful in-context learning capability of LLMs, we design the P-LLM module to predict $\tilde{\mathbf{u}}$. The P-LLM is a decoder-only transformer-based architecture [7] for prosody modeling, which uses prosody codes $\mathbf{u}$ from $\mathbf{y_p}$ as the prompt and $H_{content}$, $\tilde{H}_{content}$, and $\tilde{H}_{timbre}$ as the condition. The autoregressive prosody prediction process of P-LLM can be formulated as:

$$p\left(\tilde{\mathbf{u}} \mid \mathbf{u}, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}; \theta\right) = \prod_{t=0}^{T} p\left(\tilde{u}_t \mid \tilde{u}_{<t}, \mathbf{u}, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}; \theta\right), \tag{4}$$

where θ is the parameter of our P-LLM. Since the discrete prosody sequence $\mathbf{u}$ is phoneme-level, we directly concatenate it with $H_{content}$, $\tilde{H}_{content}$, and $\tilde{H}_{timbre}$ as the input. The P-LLM is trained in a teacher-forcing mode in the training stage via the cross-entropy loss.



(a) Zero-Shot TTS Mode (b) Speech Editing Mode

Figure 2: The inference modes for Mega-TTS. In subfigure (a), P-LLM takes information from the prompt speech to generate prosody codes for the target speech; In subfigure (b), P-LLM utilizes the contextual information from the masked speech to perform speech editing.

3.3 Speech prompting for inference

To facilitate in-context learning for various speech generation tasks, we design different speech prompting mechanisms to encourage Mega-TTS to follow the information in the speech prompt.

Inference for TTS. For zero-shot TTS, P-LLM uses $\mathbf{u}, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}$ to generate the target prosody codes $\tilde{\mathbf{u}}$ for the target speech according to Equation 4. We use the top-k random sampling scheme [17] to sample the results since we observe that the sampling-based method could increase the diversity of the generated speech. Then, we concatenate the content $\tilde{H}_{content}$, timbre $\tilde{H}_{timbre}$, and prosody $\tilde{\mathbf{u}}$ information to generate the target speech y_t using the mel decoder. Leveraging the proper inductive biases and powerful in-context learning capability of our P-LLM, the generated speech can retain not only similar timbre but also the rhythmic habits of the prompt speech. For cross-lingual TTS, $\mathbf{u}, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}$ are extracted from the prompt speech in a foreign language, and the subsequent procedure keeps the same as that of zero-shot TTS.

Inference for speech editing. In speech editing, the predicted prosody codes should achieve smooth transitions at both the left and right boundaries of the masked region. Previous works like EditSpeech [52] propose to perform left and right autoregressive inferences separately and concat the mel-spectrogram at the least L2-norm difference fusion point. However, the L2-norm difference of the mel-spectrogram is far from human perception, leading to poor audio naturalness. Since the prosody representations in Mega-TTS is discrete, we can solve the transition problem by operating on discrete prosody representations. First, we regard the area on the left side of the mask as a prompt to generate N candidate paths with top-k random sampling strategy. Secondly, the N generated paths are used as new prompts to generate the probability matrix of the area on the right side of the mask and the ground-truth prosody codes are used to obtain the probabilities of each decoding step from the probability matrix. In the third stage, we sum up the log probabilities of each decoding step for the candidate paths. Finally, we choose the path that achieves the maximum probability in the second step as the predicted result. The decoding strategy for speech editing can be formulated as follows:

$$\begin{aligned}\underset{i\in[1,N]}{\text{Max}}\ \text{Likelihood} = \underset{i\in[1,N]}{\text{Max}} \prod_{t=L}^{R} p\left(u_t^i \mid u_{<t}^i, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}; \theta\right) \\ \cdot \prod_{t=R}^{T} p\left(u_t^{gt} \mid u_{<t}^i, H_{content}, \tilde{H}_{timbre}, \tilde{H}_{content}; \theta\right),\end{aligned} \tag{5}$$

where L and R are the left and right boundaries of the mask. T is the length of the mel-spectrogram. u^i is the prosody code in the i-th candidate path. u_t^{gt} is the ground-truth prosody codes. Since our

decoding strategy considers the prosody information of the boundaries on both sides, the edited region can achieve smooth transitions.

4 Experiments

In this section, we present the evaluation results of Mega-TTS and the comparison with baselines in terms of the objective and subjective metrics.

4.1 Experimental setup

Training datasets. We use GigaSpeech [9] and WenetSpeech [65] as the training corpora, which contains 20k hours of multi-domain speeches in English and Chinese in total. Since the speech in GigaSpeech and WenetSpeech does not have speaker identities and multiple speakers may appear in a speech clip, we process the datasets with an open-source automatic speaker diarization model[6] [6, 5]. We also extract the phoneme-level alignments with the external alignment tool[7]. More information can be found in Appendix A.3.

Evaluation datasets. We employ two datasets for evaluation: 1) VCTK dataset [57], an English dataset that contains 108 speakers; 2) LibriSpeech [42] test-clean, an English dataset that contains 40 speakers. For each of these datasets, we randomly sample 10 utterances for each of the 40 speakers, resulting in a subset of 400 utterances for evaluation; Specifically, to synthesize each sample, we randomly select a different utterance of the same speaker to form the speech prompt. Note that all speakers in the evaluation datasets are unseen during training.

Model configuration. Our Mega-TTS consists of three encoders, a prosody large language model, a mel decoder, and a discriminator. The prosody encoder, timbre encoder, and mel generator consist of 5 convolutional blocks with 320 hidden size, 5 convolution 1D kernel size. The content encoder is a 4-layer Transformer [56] with 2 attention heads, 320 embedding dimensions, 1280 1D convolution filter size, and 5 convolution 1D kernel size. The duration predictor is a 3-layer 1D convolution with ReLU activation and layer normalization, which have 320 hidden size. The discriminator follows the architecture proposed in SyntaSpeech [63]. The P-LLM model is a decoder-only architecture that contains 8 Transformer layers with 8 attention heads, 512 embedding dimensions, 2048 1D convolution filter size, and 5 convolution 1D kernel size. The overall number of model parameters is 222.5M. We add more detailed model configurations in Appendix A.1.

Training and inference. In the training stage, we train Mega-TTS on 8 NVIDIA A100 GPUs, with a batch size of 30 sentences on each GPU. We use the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$ and follow the same learning rate schedule in [56]. It takes 320k steps for the VQ-GAN TTS model's training and 100K steps for the P-LLM's training until convergence. The predicted mel-spectrograms are transformed into audio samples using pre-trained HiFi-GAN V1[8] [31]. In the inference stage, we use the top-5 random sampling scheme [17] to sample diverse results.

Objective metrics. We evaluate the pitch distance and speaker similarity for zero-shot TTS. In terms of the pitch distance, we compute the average dynamic time warping (DTW) [40] distances between the pitch contours of ground-truth speech and synthesized speech. And for the cosine speaker similarity, we use the WavLM model [12] finetuned for speaker verification[9] to compute the cosine speaker similarity score between the ground-truth speech and synthesized speech. The similarity score is in the range of $[-1, 1]$, where a larger value indicates a higher similarity of input samples. In addition, we also evaluate the word error rate (WER) for cross-lingual TTS. We use the ASR system from the released HuBERT-Large model [19] to transcribe the generated speech into text. Then, the WER between the transcribed text and the original target text is measured. We use all samples in the test set for the objective evaluation. We put more information in Appendix A.4 and Appendix A.5.

[6] `https://huggingface.co/pyannote/speaker-diarization`

[7] `https://github.com/MontrealCorpusTools/Montreal-Forced-Aligner`

[8] `https://github.com/jik876/hifi-gan`

[9] `https://huggingface.co/microsoft/wavlm-base-plus-sv`

Table 2: The objective and subjective comparisons for zero-shot text-to-speech synthesis. We evaluate the audio quality, speech prosody, and speaker similarity of different systems on the VCTK and LibriSpeech test-clean datasets with 95% confidence intervals.

Dataset	Method	Subjective			Objective	
		MOS-Q (↑)	MOS-P (↑)	MOS-S (↑)	Pitch (↓)	Speaker (↑)
VCTK	Ground Truth	4.35 ± 0.11	4.48 ± 0.10	4.33 ± 0.13	-	0.915
	YourTTS [8]	4.04 ± 0.10	4.18 ± 0.09	3.76 ± 0.12	32.43	0.847
	Mega-TTS	$\mathbf{4.27 \pm 0.09}$	$\mathbf{4.32 \pm 0.11}$	$\mathbf{4.27 \pm 0.10}$	**17.45**	**0.877**
LibriSpeech	Ground Truth	4.23 ± 0.13	4.49 ± 0.11	4.29 ± 0.16	-	0.956
	YourTTS [8]	3.83 ± 0.12	4.06 ± 0.13	3.22 ± 0.21	44.05	0.909
	Mega-TTS	$\mathbf{4.08 \pm 0.17}$	$\mathbf{4.21 \pm 0.17}$	$\mathbf{3.90 \pm 0.18}$	**35.46**	**0.936**

Table 3: The comparison between Mega-TTS and VALL-E.

Method	CMOS-Q	CMOS-P	MOS-S (↑)
VALL-E [58]	-0.23	-0.27	4.06 ± 0.22
Mega-TTS	**0.00**	**0.00**	$\mathbf{4.11 \pm 0.21}$

Table 4: The MOS evaluation (↑) for speech quality, speech prosody, and speaker similarity on speech editing task on the VCTK dataset with 95% confidence intervals.

Method	MOS-Q (↑)	MOS-P (↑)	MOS-S (↑)
EditSpeech [52]	3.57 ± 0.12	3.87 ± 0.14	3.93 ± 0.14
A^3T [3]	3.73 ± 0.13	3.96 ± 0.14	3.97 ± 0.12
Mega-TTS	$\mathbf{3.81 \pm 0.14}$	$\mathbf{4.11 \pm 0.14}$	$\mathbf{4.36 \pm 0.16}$

Subjective metrics. We conduct the MOS (mean opinion score) and CMOS (comparative mean opinion score) evaluation on the test set to measure the audio naturalness via Amazon Mechanical Turk. We keep the text content and prompt speech consistent among different models to exclude other interference factors. We randomly choose 50 samples from the test set of each dataset for the subjective evaluation and each audio is listened to by at least 20 testers. We analyze the MOS in three aspects: MOS-Q (Quality: clarity, high-frequency, and original timbre reconstruction), MOS-P (Prosody: naturalness of pitch, energy, and duration), and MOS-S (Speaker similarity). We also analyze the CMOS in terms of audio quality and speech prosody. We tell the tester to focus on one corresponding aspect and ignore the other aspect when scoring. We put more information about the subjective evaluation in Appendix A.2.

4.2 Results of zero-shot synthesis

We compare the zero-shot synthesis performance of Mega-TTS with baseline systems, including: 1)YourTTS [8], a powerful zero-shot TTS model trained on 1k hours of speech dataset. We use the official code and released checkpoint[10]; 2) VALL-E, a large-scale zero-shot TTS model using the audio codec model to generate discrete speech codes and LLM to generate them. For VALL-E, we directly download the first 16 utterances from the VALL-E demo page. The audio samples consist of 8 samples from LibriSpeech and 8 samples from VCTK[11]. As shown in Table 2, Mega-TTS significantly outperforms YourTTS in terms of audio quality and speech prosody. And in terms of speaker similarity, Mega-TTS significantly outperforms YourTTS with +0.51 MOS-S on VCTK and +0.68 MOS-S on LibriSpeech, demonstrating the effectiveness of Mega-TTS in zero-shot scenarios. Besides, as shown in Table 3, Mega-TTS outperforms VALL-E in all metrics. It can be seen that Mega-TTS is able to generate more natural speeches than VALL-E, demonstrating the effectiveness of introducing intrinsic inductive biases. To further investigate the performance of disentanglement, we also visualize the distribution of the timbre and prosody representations in Appendix C.

[10] `https://github.com/Edresson/YourTTS`

[11] VALL-E does not release its code officially. The unofficial implementations and our implementation are deficient, which would make us difficult to fairly compare our system with VALL-E.

Table 5: The comparisons for cross-lingual text-to-speech synthesis with 95% confidence intervals. We also measure the word error rate (WER) and speaker similarity score for the objective evaluations.

Method	Subjective			Objective	
	MOS-Q (↑)	MOS-P (↑)	MOS-S (↑)	WER (↓)	Speaker (↑)
YourTTS [8]	3.65 ± 0.21	3.92 ± 0.18	3.32 ± 0.27	7.59%	0.883
VALL-E X [67]	3.73 ± 0.17	3.97 ± 0.18	3.81 ± 0.16	-	-
Mega-TTS	$\mathbf{3.85 \pm 0.17}$	$\mathbf{4.08 \pm 0.19}$	$\mathbf{3.86 \pm 0.18}$	**3.04%**	**0.919**

Table 6: The comparison of robustness between Mega-TTS and other systems on the 50 particularly hard sentences. Each kind of word error is counted at once per sentence.

Method	Repeats	Skips	Error Sentences	Error Rate
Tacotron [59]	10	16	22	44%
VALL-E [58]	8	11	14	28%
FastSpeech [48]	0	0	0	0%
Mega-TTS	0	0	0	0%

4.3 Results of zero-shot speech editing

We compare the quality of generated audio samples of our Mega-TTS with SOTA speech editing baselines, including 1) EditSpeech [52]; 2) A^3T [3]. Since the text content of the generated speech has been edited in the speech editing evaluation, the ground truth is missing. Therefore, we only conduct the subjective evaluation. We manually define modification operations (i.e., insertion, replacement, and deletion) of the test samples. We then conduct the experiments on the VCTK dataset. We evaluate the audio quality, speech prosody, and speaker similarity for each audio sample. The results are presented Table 4. It can be seen that Mega-TTS achieves the highest perceptual quality, prosody, and speaker similarity score, which demonstrates the effectiveness of our proposed speech prompting mechanism for speech editing and the powerful in-context learning capability of Mega-TTS.

4.4 Results of zero-shot cross-lingual TTS

To compare Mega-TTS with the zero-shot cross-lingual TTS models VALL-E X [67], we directly download the utterances from the VALL-E X demo page, which consists of 6 speech pairs from LibriSpeech, EMIME, and AISHELL-3. Since YourTTS [8] is built only for English TTS, we evaluate the performance of English TTS with Chinese samples as prompts. The results are listed in Table 5. It can be seen that Mega-TTS surpasses VALL-E X in terms of audio quality, speech prosody, and speaker similarity scores, which further demonstrates the superiority of introducing proper inductive biases to different speech attributes. For objective evaluations, we use all of the text samples in the LibriSpeech test-clean set as the target sentences and randomly select one audio from AISHELL-3 as the speech prompt for each target sentence. The results show that Mega-TTS achieves a significantly lower WER than YourTTS, demonstrating the effectiveness of our method.

4.5 Results of robustness evaluation

To further evaluate the robustness of the proposed model, we adopt the 50 particularly hard sentences following FastSpeech [48]. As shown in Table 6, Tacotron [59] and VALL-E [58] show poor robustness on these complicated sentences. As a comparison, our Mega-TTS shows equivalent robustness to the non-autoregressive models, such as FastSpeech [48], without any repeat or skip issues. It can be seen that directly modeling the discrete speech tokens with LLMs like VALL-E [58] would cause robustness issues. As a comparison, Mega-TTS not only leverages the in-context learning capability of LLMs, but also maintains good robustness by introducing the proper inductive bias to each speech component.

5 Conclusion

In this paper, we proposed Mega-TTS, which aims to introduce proper inductive biases into large-scale zero-shot TTS systems. We disentangle speech into different attributes (i.e., content, timbre, prosody, and phase) and model different attributes in different ways. We train Mega-TTS with 20K hours of multi-domain speech data and evaluate its performance on unseen datasets. Our experimental results on three speech synthesis tasks show that Mega-TTS outperforms state-of-the-art zero-shot TTS models regarding audio quality, speech prosody, speaker similarity, and robustness. Due to limited page space, we discuss the limitations and future works in Appendix F and the broader impacts in Appendix G.

References

[1] Sercan Arik, Jitong Chen, Kainan Peng, Wei Ping, and Yanqi Zhou. Neural voice cloning with a few samples. *Advances in neural information processing systems*, 31, 2018.

[2] Sercan Ö Arık, Mike Chrzanowski, Adam Coates, Gregory Diamos, Andrew Gibiansky, Yongguo Kang, Xian Li, John Miller, Andrew Ng, Jonathan Raiman, et al. Deep voice: Real-time neural text-to-speech. In *International conference on machine learning*, pages 195–204. PMLR, 2017.

[3] He Bai, Renjie Zheng, Junkun Chen, Mingbo Ma, Xintong Li, and Liang Huang. A^3t: Alignment-aware acoustic and text pretraining for speech synthesis and editing. In *International Conference on Machine Learning*, pages 1399–1411. PMLR, 2022.

[4] Zalán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Olivier Teboul, David Grangier, Marco Tagliasacchi, and Neil Zeghidour. Audiolm: a language modeling approach to audio generation. *arXiv preprint arXiv:2209.03143*, 2022.

[5] Hervé Bredin and Antoine Laurent. End-to-end speaker segmentation for overlap-aware resegmentation. In *Proc. Interspeech 2021*, 2021.

[6] Hervé Bredin, Ruiqing Yin, Juan Manuel Coria, Gregory Gelly, Pavel Korshunov, Marvin Lavechin, Diego Fustes, Hadrien Titeux, Wassim Bouaziz, and Marie-Philippe Gill. pyannote.audio: neural building blocks for speaker diarization. In *ICASSP 2020, IEEE International Conference on Acoustics, Speech, and Signal Processing*, 2020.

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[8] Edresson Casanova, Julian Weber, Christopher D Shulby, Arnaldo Candido Junior, Eren Gölge, and Moacir A Ponti. Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone. In *International Conference on Machine Learning*, pages 2709–2720. PMLR, 2022.

[9] Guoguo Chen, Shuzhou Chai, Guanbo Wang, Jiayu Du, Wei-Qiang Zhang, Chao Weng, Dan Su, Daniel Povey, Jan Trmal, Junbo Zhang, et al. Gigaspeech: An evolving, multi-domain asr corpus with 10,000 hours of transcribed audio. *arXiv preprint arXiv:2106.06909*, 2021.

[10] Jiawei Chen, Xu Tan, Jian Luan, Tao Qin, and Tie-Yan Liu. Hifisinger: Towards high-fidelity neural singing voice synthesis. *arXiv preprint arXiv:2009.01776*, 2020.

[11] Mingjian Chen, Xu Tan, Bohan Li, Yanqing Liu, Tao Qin, Sheng Zhao, and Tie-Yan Liu. Adaspeech: Adaptive text to speech for custom voice. *arXiv preprint arXiv:2103.00993*, 2021.

[12] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, et al. Wavlm: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.

[13] Yutian Chen, Yannis Assael, Brendan Shillingford, David Budden, Scott Reed, Heiga Zen, Quan Wang, Luis C Cobo, Andrew Trask, Ben Laurie, et al. Sample efficient adaptive text-to-speech. *arXiv preprint arXiv:1809.10460*, 2018.

[14] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*, 2022.

[15] Jeff Donahue, Sander Dieleman, Mikolaj Binkowski, Erich Elsen, and Karen Simonyan. End-to-end adversarial text-to-speech. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.

[16] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021.

[17] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 889–898, 2018.

[18] Karan Goel, Albert Gu, Chris Donahue, and Christopher Ré. It's raw! audio generation with state-space models. In *International Conference on Machine Learning*, pages 7616–7633. PMLR, 2022.

[19] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

[20] Rongjie Huang, Max WY Lam, Jun Wang, Dan Su, Dong Yu, Yi Ren, and Zhou Zhao. Fastdiff: A fast conditional diffusion model for high-quality speech synthesis. *arXiv preprint arXiv:2204.09934*, 2022.

[21] Rongjie Huang, Yi Ren, Jinglin Liu, Chenye Cui, and Zhou Zhao. Generspeech: Towards style transfer for generalizable out-of-domain text-to-speech synthesis. *arXiv preprint arXiv:2205.07211*, 2022.

[22] Rongjie Huang, Zhou Zhao, Huadai Liu, Jinglin Liu, Chenye Cui, and Yi Ren. Prodiff: Progressive fast diffusion model for high-quality text-to-speech. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 2595–2605, 2022.

[23] Sung-Feng Huang, Chyi-Jiunn Lin, Da-Rong Liu, Yi-Chen Chen, and Hung-yi Lee. Meta-tts: Meta-learning for few-shot speaker adaptive text-to-speech. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:1558–1571, 2022.

[24] Myeonghun Jeong, Hyeongju Kim, Sung Jun Cheon, Byoung Jin Choi, and Nam Soo Kim. Diff-tts: A denoising diffusion model for text-to-speech. *arXiv preprint arXiv:2104.01409*, 2021.

[25] Ye Jia, Yu Zhang, Ron Weiss, Quan Wang, Jonathan Shen, Fei Ren, Patrick Nguyen, Ruoming Pang, Ignacio Lopez Moreno, Yonghui Wu, et al. Transfer learning from speaker verification to multispeaker text-to-speech synthesis. *Advances in neural information processing systems*, 31, 2018.

[26] Minki Kang, Dongchan Min, and Sung Ju Hwang. Any-speaker adaptive text-to-speech synthesis with diffusion models. *arXiv preprint arXiv:2211.09383*, 2022.

[27] Eugene Kharitonov, Damien Vincent, Zalán Borsos, Raphaël Marinier, Sertan Girgin, Olivier Pietquin, Matt Sharifi, Marco Tagliasacchi, and Neil Zeghidour. Speak, read and prompt: High-fidelity text-to-speech with minimal supervision. *arXiv preprint arXiv:2302.03540*, 2023.

[28] Heeseung Kim, Sungwon Kim, and Sungroh Yoon. Guided-tts: A diffusion model for text-to-speech via classifier guidance. In *International Conference on Machine Learning*, pages 11119–11133. PMLR, 2022.

[29] Jaehyeon Kim, Sungwon Kim, Jungil Kong, and Sungroh Yoon. Glow-tts: A generative flow for text-to-speech via monotonic alignment search. *Advances in Neural Information Processing Systems*, 33:8067–8077, 2020.

[30] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *International Conference on Machine Learning*, pages 5530–5540. PMLR, 2021.

[31] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis. *Advances in Neural Information Processing Systems*, 33:17022–17033, 2020.

[32] Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. Diffwave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

[33] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi. Audiogen: Textually guided audio generation. *arXiv preprint arXiv:2209.15352*, 2022.

[34] Yoonhyung Lee, Joongbo Shin, and Kyomin Jung. Bidirectional variational inference for non-autoregressive text-to-speech. In *International Conference on Learning Representations*, 2021.

[35] Naihan Li, Shujie Liu, Yanqing Liu, Sheng Zhao, and Ming Liu. Neural speech synthesis with transformer network. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 6706–6713, 2019.

[36] Jinglin Liu, Chengxi Li, Yi Ren, Feiyang Chen, and Zhou Zhao. Diffsinger: Singing voice synthesis via shallow diffusion mechanism. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 11020–11028, 2022.

[37] Yanqing Liu, Ruiqing Xue, Lei He, Xu Tan, and Sheng Zhao. Delightfultts 2: End-to-end speech synthesis with adversarial vector-quantized auto-encoders. *arXiv preprint arXiv:2207.04646*, 2022.

[38] Xudong Mao, Qing Li, Haoran Xie, Raymond YK Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2794–2802, 2017.

[39] Chenfeng Miao, Shuang Liang, Minchuan Chen, Jun Ma, Shaojun Wang, and Jing Xiao. Flow-tts: A non-autoregressive network for text to speech based on flow. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7209–7213. IEEE, 2020.

[40] Meinard Müller. Dynamic time warping. *Information retrieval for music and motion*, pages 69–84, 2007.

[41] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

[42] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: an asr corpus based on public domain audio books. In *2015 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 5206–5210. IEEE, 2015.

[43] Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, and Mikhail Kudinov. Grad-tts: A diffusion probabilistic model for text-to-speech. In *International Conference on Machine Learning*, pages 8599–8608. PMLR, 2021.

[44] Kaizhi Qian, Yang Zhang, Shiyu Chang, Xuesong Yang, and Mark Hasegawa-Johnson. Autovc: Zero-shot voice style transfer with only autoencoder loss. In *International Conference on Machine Learning*, pages 5210–5219. PMLR, 2019.

[45] Yi Ren, Chenxu Hu, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. Fastspeech 2: Fast and high-quality end-to-end text to speech. *arXiv preprint arXiv:2006.04558*, 2020.

[46] Yi Ren, Ming Lei, Zhiying Huang, Shiliang Zhang, Qian Chen, Zhijie Yan, and Zhou Zhao. Prosospeech: Enhancing prosody with quantized vector pre-training in text-to-speech. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7577–7581. IEEE, 2022.

[47] Yi Ren, Jinglin Liu, and Zhou Zhao. Portaspeech: Portable and high-quality generative text-to-speech. *Advances in Neural Information Processing Systems*, 34, 2021.

[48] Yi Ren, Yangjun Ruan, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. Fastspeech: Fast, robust and controllable text to speech. *Advances in neural information processing systems*, 32, 2019.

[49] Jonathan Shen, Ruoming Pang, Ron J Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, Rj Skerrv-Ryan, et al. Natural tts synthesis by conditioning wavenet on mel spectrogram predictions. In *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4779–4783. IEEE, 2018.

[50] Kai Shen, Zeqian Ju, Xu Tan, Yanqing Liu, Yichong Leng, Lei He, Tao Qin, Sheng Zhao, and Jiang Bian. Naturalspeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers. *arXiv preprint arXiv:2304.09116*, 2023.

[51] Yao Shi, Hui Bu, Xin Xu, Shaoji Zhang, and Ming Li. Aishell-3: A multi-speaker mandarin tts corpus and the baselines. *arXiv preprint arXiv:2010.11567*, 2020.

[52] Daxin Tan, Liqun Deng, Yu Ting Yeung, Xin Jiang, Xiao Chen, and Tan Lee. Editspeech: A text based speech editing system using partial inference and bidirectional fusion. In *2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 626–633. IEEE, 2021.

[53] Xu Tan, Tao Qin, Frank Soong, and Tie-Yan Liu. A survey on neural speech synthesis. *arXiv preprint arXiv:2106.15561*, 2021.

[54] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

[55] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[57] Christophe Veaux, Junichi Yamagishi, Kirsten MacDonald, et al. Superseded-cstr vctk corpus: English multi-speaker corpus for cstr voice cloning toolkit. 2016.

[58] Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, et al. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*, 2023.

[59] Yuxuan Wang, RJ Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, et al. Tacotron: Towards end-to-end speech synthesis. *arXiv preprint arXiv:1703.10135*, 2017.

[60] Yuxuan Wang, Daisy Stanton, Yu Zhang, RJ-Skerry Ryan, Eric Battenberg, Joel Shor, Ying Xiao, Ye Jia, Fei Ren, and Rif A Saurous. Style tokens: Unsupervised style modeling, control and transfer in end-to-end speech synthesis. In *International Conference on Machine Learning*, pages 5180–5189. PMLR, 2018.

[61] Yihan Wu, Xu Tan, Bohan Li, Lei He, Sheng Zhao, Ruihua Song, Tao Qin, and Tie-Yan Liu. Adaspeech 4: Adaptive text to speech in zero-shot scenarios. *arXiv preprint arXiv:2204.00436*, 2022.

[62] Ryuichi Yamamoto, Eunwoo Song, and Jae-Min Kim. Parallel wavegan: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6199–6203. IEEE, 2020.

[63] Zhenhui Ye, Zhou Zhao, Yi Ren, and Fei Wu. Syntaspeech: syntax-aware generative adversarial text-to-speech. *arXiv preprint arXiv:2204.11792*, 2022.

[64] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021.

[65] Binbin Zhang, Hang Lv, Pengcheng Guo, Qijie Shao, Chao Yang, Lei Xie, Xin Xu, Hui Bu, Xiaoyu Chen, Chenchen Zeng, et al. Wenetspeech: A 10000+ hours multi-domain mandarin corpus for speech recognition. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6182–6186. IEEE, 2022.

[66] Chen Zhang, Yi Ren, Xu Tan, Jinglin Liu, Kejun Zhang, Tao Qin, Sheng Zhao, and Tie-Yan Liu. Denoispeech: Denoising text to speech with frame-level noise modeling. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7063–7067. IEEE, 2021.

[67] Ziqiang Zhang, Long Zhou, Chengyi Wang, Sanyuan Chen, Yu Wu, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, et al. Speak foreign languages with your own voice: Cross-lingual neural codec language modeling. *arXiv preprint arXiv:2303.03926*, 2023.

Appendices

A Detailed Experimental Settings

A.1 Model Configurations

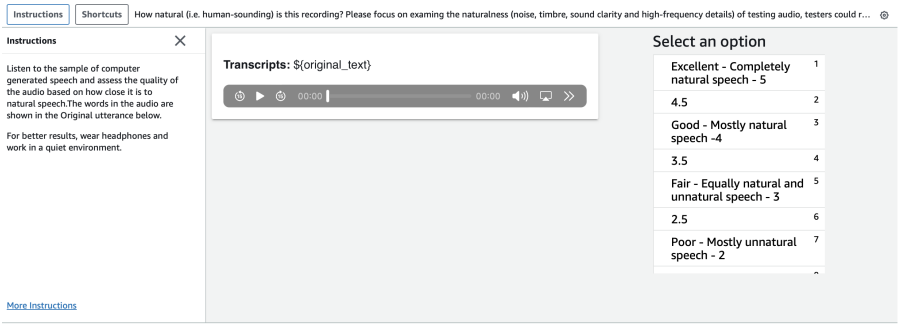
We list the model hyper-parameters of Mega-TTS in Table 7.

Table 7: Hyperparameters of Mega-TTS models.

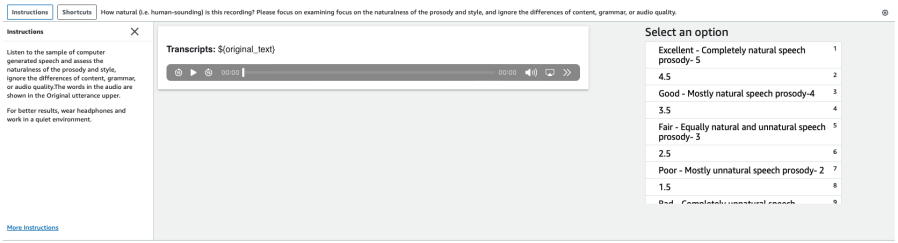
Hyper-parameter		Value
Prosody Encoder	Encoder Layers	5
	Hidden Size	320
	Conv1D Kernel	5
	VQ Embedding Size	2048
	VQ Embedding Channel	256
Content Encoder	Phoneme Embedding Size	320
	Encoder Layers	4
	Hidden Size	320
	Kernel Size	5
	Filter Size	1280
Timbre Encoder	Encoder Layers	5
	Hidden Size	320
	Conv1D Kernel	31
Mel Decoder	Decoder Layers	5
	Hidden Size	320
	Conv1D Kernel	5
P-LLM	Decoder Layers	8
	Hidden Size	512
	Decoder Kernel Size	5
	Decoder channel size	2048
	Prosody Code Embedding Size	2050
	Attention Headss	8
	Number of Contextual Sentences	7
Multi-Length Discriminator	Number of Discriminators	3
	Window Size	32, 64, 128
	Conv2D Layers	3
	Hidden Size	192
Total Number of Parameters		222.5M

A.2 Details in Subjective Evaluation

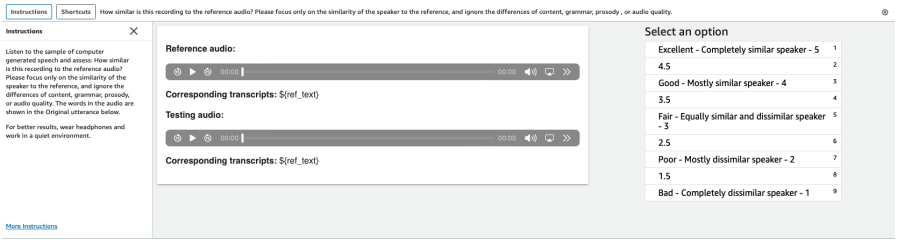
We perform the audio quality, speech prosody, and speaker similarity evaluations on Amazon Mechanical Turk (MTurk). For each dataset, we randomly select 50 samples from the test set and use the TTS systems to generate the audio samples. Each audio has been listened to by at least 20 listeners. For MOS, each tester is asked to evaluate the subjective score of a sentence on a 1-5 Likert scale. For CMOS, listeners are asked to compare pairs of audio generated by systems A and B, indicating which of the two audio they prefer, and choose one of the following scores according to the degree of superiority: 0 indicating no difference, 1 indicating source slightly better, 2 indicating source mostly better and 3 indicating source completely better. For audio quality evaluation (MOS-Q and CMOS-Q), we tell listeners to "*Please focus on the audio quality and ignore other factors*". For prosody evaluations (MOS-P and CMOS-P), we tell listeners to "*Please focus on the prosody and style, and ignore the differences of grammar, audio quality, or other factors.* ". For speaker similarity



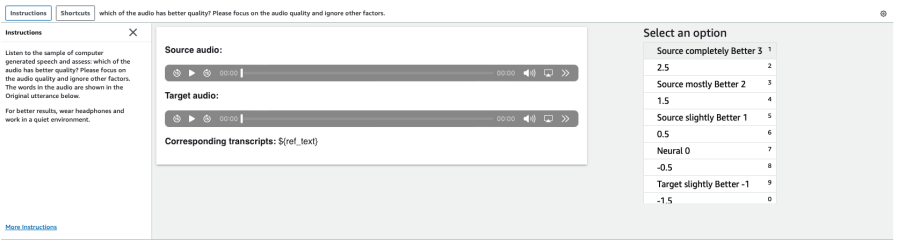
(a) Screenshot of MOS-Q testing.



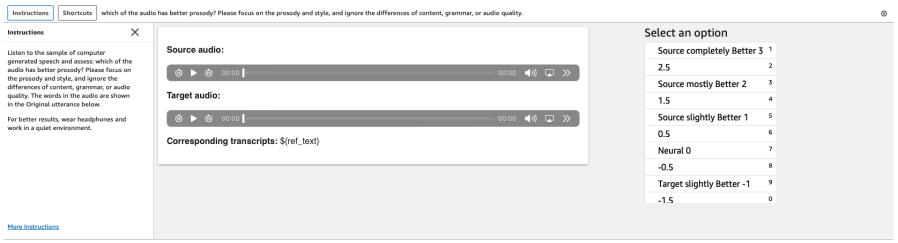
(b) Screenshot of MOS-P testing.



(c) Screenshot of MOS-S testing.



(d) Screenshot of CMOS-Q testing.



(e) Screenshot of CMOS-P testing.

Figure 3: Screenshots of MOS and CMOS testings for audio quality, speech prosody, and speaker similarity evaluations.

evaluations (MOS-S), we tell listeners to " *Please focus only on the similarity of the speaker to the reference, and ignore the differences of content, grammar, prosody, audio quality, or other factors.*".

The screenshots of instructions for testers are shown in Figure 3. We paid $12 to participants hourly and totally spent about $1000 on participant compensation. We tell the participants that the data will be used in scientific research.

A.3 Details of Speaker Diarization Model

To obtain the speaker information from GigaSpeech and WenetSpeech, we use a released automatic speaker diarization model called *pyannote.audio*[12], which achieves DER=11.24% on the VoxConverse dataset and DER=14.09% on the AISHELL-4 dataset. We only assign the speaker ID to the audio clip when its activation score is higher than 70% and abandon other audio clips. We also abandon the audio clips that contain multiple speakers speaking simultaneously.

A.4 Details of Speaker Similarity Model

To measure the speaker similarity, we use the WavLM [12] model finetuned for speaker verification from `https://huggingface.co/microsoft/wavlm-base-plus-sv` to extract the speaker embedding. Then the cosine similarity between the synthesized speech's speaker embedding and the ground-truth speech's speaker embedding is calculated as the speaker similarity score. The WavLM model is pretrained on 94,000 hours of speech data and finetuned on the VoxCeleb1 dataset using an X-Vector head with an Additive Margin Softmax loss, which achieves 0.84%, 0.928%, and 1.758% EER (Equal Error Rate) on the Vox1-O, Vox1-E, and Vox1-H trial lists.

A.5 Details of ASR Model

To measure the audio quality and speech intelligibility for cross-lingual TTS systems, we evaluate the word error rate (WER) metric. We use the finetuned HuBERT-Large model to transcribe the synthesized speech into text and calculate the WER between the transcribed text and the original target text. The finetuned HuBERT-Large model from `https://huggingface.co/facebook/hubert-large-ls960-ft` is finetuned on 960h of Librispeech and achieves 1.5%, 3.0%, 1.9%, and 3.3% WER on the dev-clean, dev-other, test-clean, and test-other set of Librispeech.

A.6 Error Bars and Random Seeds

For the subjective evaluations, we report confidence intervals of the results of MOS tests in Table 2, Table 3, Table 4, and Table 5. For the objective evaluations, we ran the experiments 10 times with 10 different random seeds ($[1234, 1111, 2222, 3333, 4444, 5555, 6666, 7777, 8888, 9999]$) and obtained the averaged results.

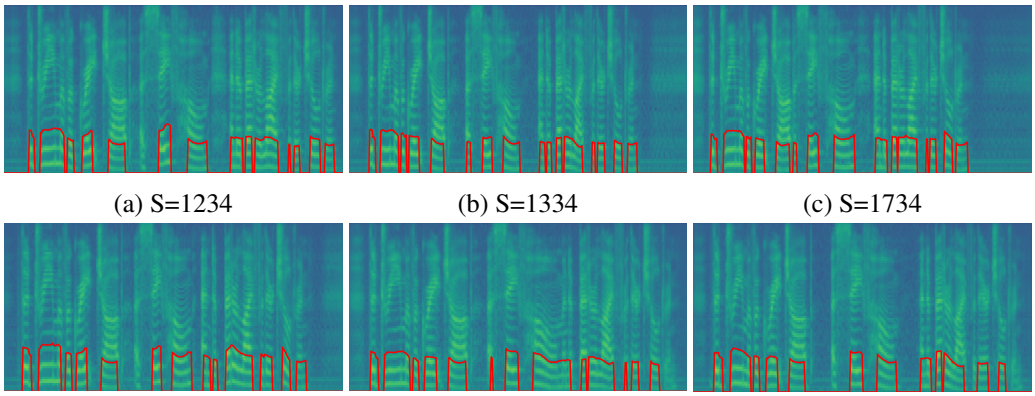
B Visualizations of Mel-Spectrograms

We put more visualizations of mel-spectrograms with different random seeds in Figure 4. We can see that with different random seeds, Mega-TTS can generate diverse results that have different prosody and frequency details.

C Visualization of Representations

To validate the effectiveness of disentanglement for speech components in Section 3.1, we adopt T-SNE [55] to visualize timbre embedding and prosody embedding for unseen speakers on the VCTK dataset. We randomly select 10 speakers and directly use the encoders proposed in Section 3.1 to extract the timbre and prosody information from their audio samples. The results are shown in Figure 5 and Figure 6. It can be seen that the timbre embeddings are ideally located according to the speaker ID. However, the prosody embeddings of different speakers have similar distributions. It shows that our proposed prosody and timbre encoders are able to disentangle the corresponding representations from the mel-spectrograms, which further ensures the effectiveness of our P-LLM.

[12] `https://huggingface.co/pyannote/speaker-diarization`



(a) S=1234 (b) S=1334 (c) S=1734

(d) S=3234 (e) S=4999 (f) S=9222

Figure 4: Visualizations of the mel-spectrograms generated with different random seeds S.

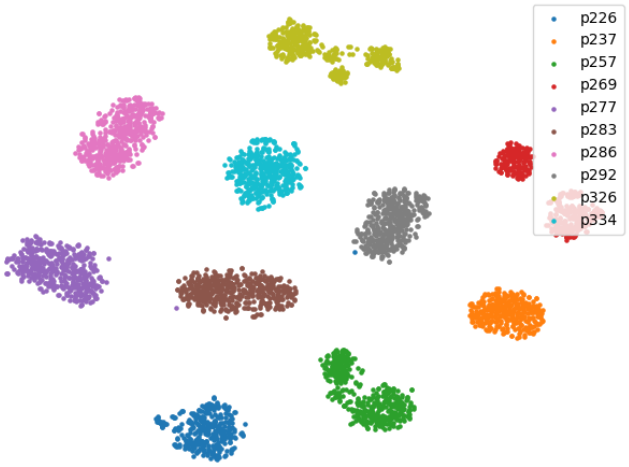


Figure 5: The T-SNE visualization of timbre embeddings for 10 unseen speakers on the VCTK dataset.

Table 8: The disentanglement performance evaluation for VQGAN-based TTS model with different VQ hyperparameters.

Channel Size * Embedding Size	Pitch (↓)	Speaker (↑)
64*512	73.82	0.719
256*2048	**49.30**	**0.941**
1024*4096	78.84	0.707

D Hyperparameter Selection for the Information Bottleneck

In this section, we describe the details of the hyperparameter selection for the information bottleneck proposed in Section 3.1. The information bottleneck of Mega-TTS mainly contains two key hyperparameters: the channel size and the embedding size of the vector quantization (VQ) layer. When the channel size and the embedding size are too small or large, the performance of disentanglement will be poor. Therefore, we should carefully select these hyperparameters. We train the VQGAN-based TTS models with different VQ hyperparameters and evaluate their pitch distance and speaker similarity following Section 4. Differently, we use the proposed encoders to extract the timbre, content, and prosody embeddings of the test samples. Then, we randomly shuffle the timbre embedding sequence

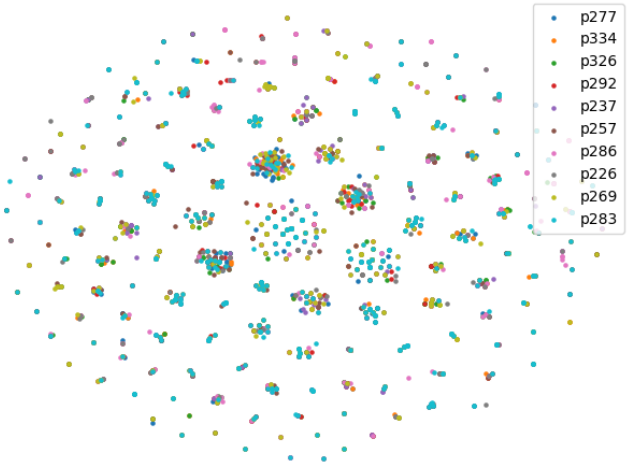


Figure 6: The T-SNE visualization of prosody embeddings for 10 unseen speakers on the VCTK dataset.

Table 9: The performance of Mega-TTS using different sizes of the training data.

Dataset Usage	Total Time (hours)	Pitch (↓)	Speaker (↑)	Duration (↓)
GiGaSpeech	10K	**36.50**	**0.935**	**62.61**
LibriSpeech	960	43.90	0.915	69.85
VCTK	44	81.33	0.828	82.39

Table 10: The performance of Mega-TTS with different hidden sizes of P-LLM.

Hidden Size of P-LLM	Pitch (↓)	Speaker (↑)
128	82.24	0.917
256	71.74	0.920
512	**35.46**	**0.936**

and reconstruct the mel-spectrogram with the original content, original prosody, and shuffled timbre information. We calculate the pitch distance between the ground-truth speech and the generated speech, but we calculate the speaker similarity between the shuffled ground-truth speech and the generated speech. As shown in Table 8, when the channel size is 256 and the embedding size is 2048, the VQGAN-based TTS model shows the best pitch accuracy and speaker similarity, i.e., the disentanglement performance is the best.

E Ablation Studies of Dataset Size and Model Size

In this section, we evaluate the influences of the training dataset size and model size on the zero-shot TTS task for Mega-TTS. We evaluate the pitch distance, speaker similarity, and the average absolute duration error in milliseconds on the LibriSpeech test-clean set. As shown in Table 9, when the dataset size grows, the zero-shot performance of Mega-TTS is significantly improved. Moreover, from Table 10, we can see that when the hidden size of P-LLM grows, the pitch distance significantly drops, demonstrating that the in-context learning capability of P-LLM can be greatly improved by the size of the model.

F Limitations and Future Works

Although achieving superior performance on various zero-shot speech synthesis tasks, Mega-TTS still suffers from two main limitations.

Data coverage. Although we use 20K hours of multi-domain data for training, our model still cannot cover everyone's voice. Especially for some speakers with extremely heavy accents, our model cannot imitate their speaking style very well. In the future, we will further scale up the training data to 200K hours to further improve the performance of the model.

Reconstruction Robustness. Although the reconstruction quality of the proposed VQGAN-based TTS model is satisfying on the clean dataset, it will be influenced by the background music or the extremely loud reverberation. In future work, we will explore a new model structure that is more robust against the acoustic environment noises.

G Broader Impacts

Mega-TTS improves the quality and efficiency of zero-shot speech synthesis, which makes it easier for people to synthesize personalized speeches. In most cases, people will utilize this technique to facilitate movies, games, podcasts, and other services only. However, it may carry potential risks in misuse of the model, such as spoofing voice or other deepfake-related usages. To handle this, potential solutions like building a corresponding deepfake detection model should be considered. We also plan to include restrictions in the open-source license of the Mega-TTS project to prevent the misuse of the model.